```
# Import libraries
import numpy as np
import pandas as pd
import seaborn as sns
import  matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from imblearn.over_sampling import SMOTE
```

```
# import all models
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
```

```
from sklearn.naive_bayes import GaussianNB
# from lightgbm import LGBMClassifier
```

```
# impot pipeline
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
```

```
# import metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
```

```
import imblearn
print(imblearn.__version__)
```

```
0.14.1
```

Load Dataset

```
# Load Data

df = pd.read_csv('diabetes.csv')
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps: Generate code with df | New interactive sheet

Information about Dataset

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
```

```
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

Check Missing values

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Statistics summary

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outc
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000

3. Fix Incorrect Zero Values

```
import warnings
warnings.filterwarnings('ignore')


cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in cols:
    df[col] = df[col].replace(0, np.nan)
    df[col].fillna(df[col].median(), inplace=True)

df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

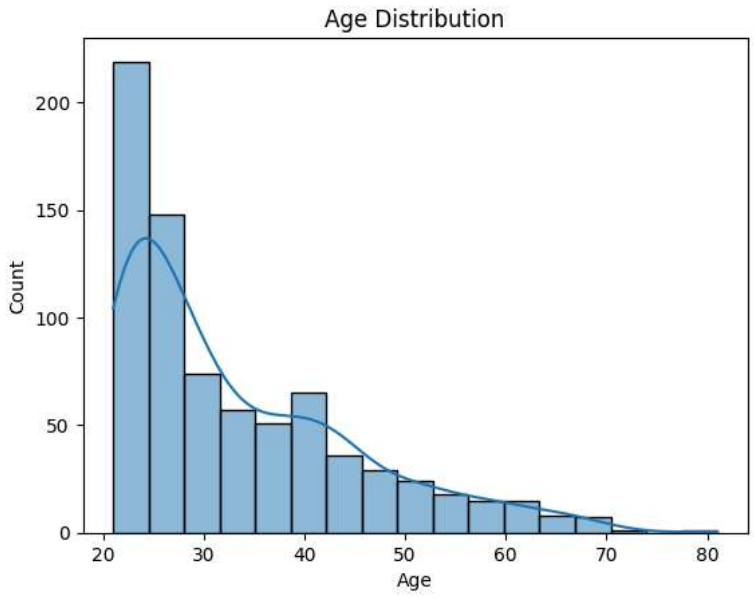
4. Feature Engineering

```
df['BMI_Glucose'] = df['BMI'] * df['Glucose']
df['AgeGroup'] = pd.cut(df['Age'], bins=[20,30,40,50,60,85], labels=[1,2,3,4,5])
df['AgeGroup'] = df['AgeGroup'].astype('Int64')

df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	BMI_Glucose	AgeGrou
0	6	148.0	72.0	35.0	125.0	33.6	0.627	50	1	4972.8	
1	1	85.0	66.0	29.0	125.0	26.6	0.351	31	0	2261.0	
2	8	183.0	64.0	29.0	125.0	23.3	0.672	32	1	4263.9	
3	1	89.0	66.0	23.0	94.0	28.1	0.167	21	0	2500.9	
4	0	137.0	40.0	35.0	168.0	43.1	2.288	33	1	5904.7	

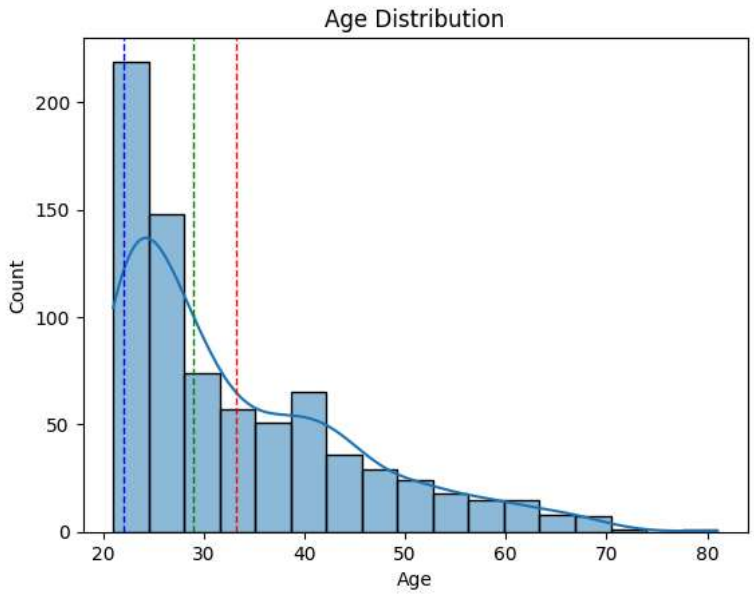
Next steps: Generate code with df | New interactive sheet

```
sns.histplot(df['Age'], kde=True)
plt.title('Age Distribution')
plt.show()
```




```
# plot mean, median and mode of the age column using sns
sns.histplot(df['Age'], kde=True)
plt.axvline(df['Age'].mean(), color='red', linestyle='dashed', linewidth=1)
```

```
plt.axvline(df['Age'].median(), color='green', linestyle='dashed', linewidth=1)
plt.axvline(df['Age'].mode()[0], color='blue', linestyle='dashed', linewidth=1)
plt.title('Age Distribution')
plt.show()
# Print the value of mean, median and mode of the age column
print(f'Mean: {df["Age"].mean()}')
print(f'Median: {df["Age"].median()}')
print(f'Mode: {df["Age"].mode()[0]}')
```




```
Mean: 33.240885416666664
Median: 29.0
Mode: 22
```

```
# Find the values count of the age column grouping by sex column

df.groupby('Outcome')['Age'].value_counts()
```

		count
Outcome	**Age**	
0	**22**	61
	21	58
	24	38
	25	34
	23	31
...	**...**	...
1	**55**	1
	57	1
	61	1
	67	1
	70	1

96 rows × 1 columns

dtype: int64

```
# Find the mean of all columns groupbt by outcome column
df.groupby('Outcome').mean()
```

Outcome	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	BMI_Gl
0	3.298000	110.682000	70.920000	27.726000	127.792000	30.885600	0.429734	31.190000	3437.6
1	4.865672	142.130597	75.123134	31.686567	164.701493	35.383582	0.550500	37.067164	5039.6

```
df['Outcome'].value_counts()
```

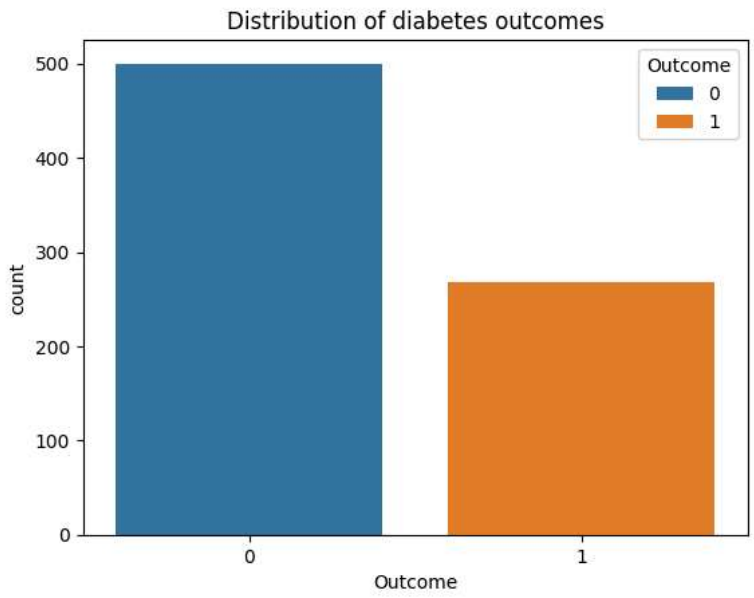
Outcome	count
0	500
1	268

dtype: int64

Visualization

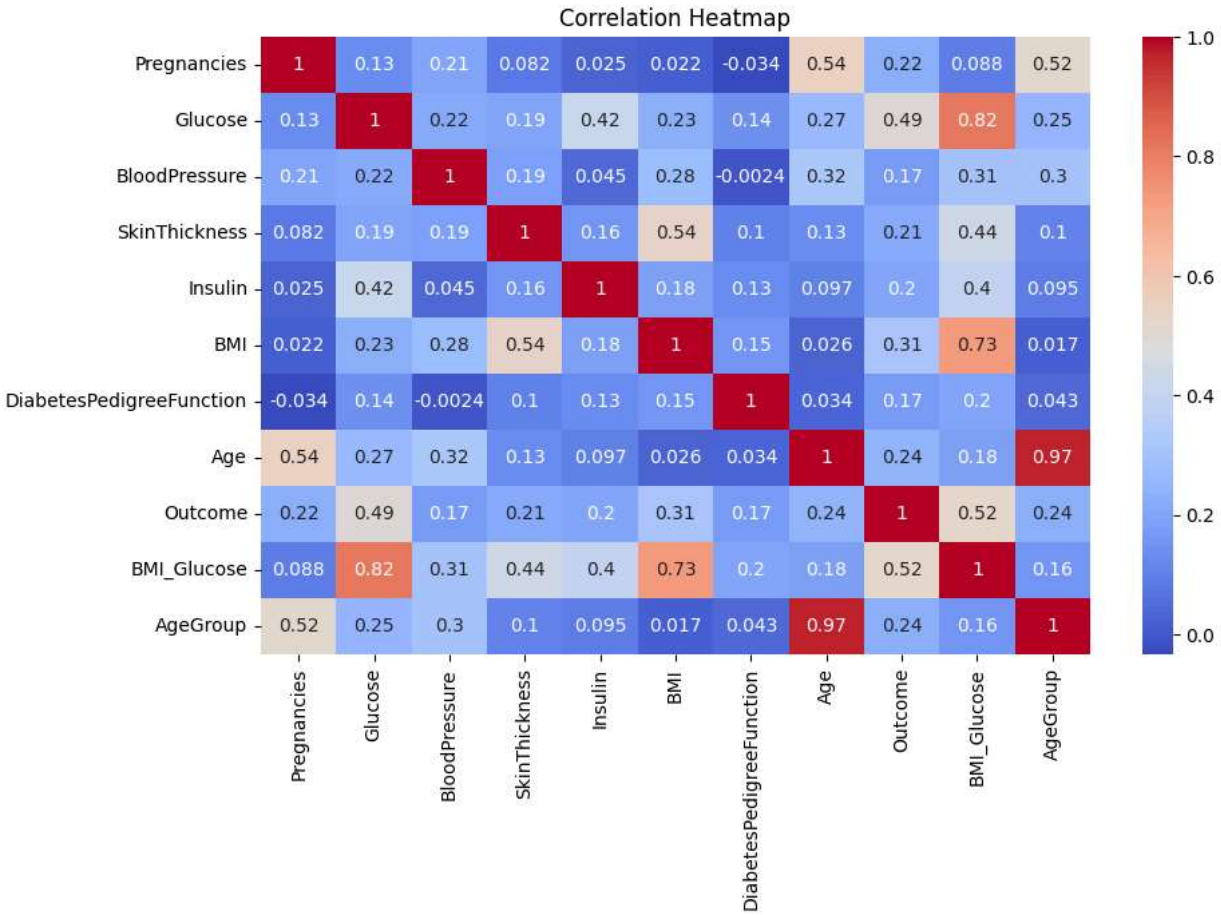
Count Outcome

```
sns.countplot(data=df, x = 'Outcome', hue= 'Outcome')
plt.title('Distribution of diabetes outcomes')
plt.show()
```



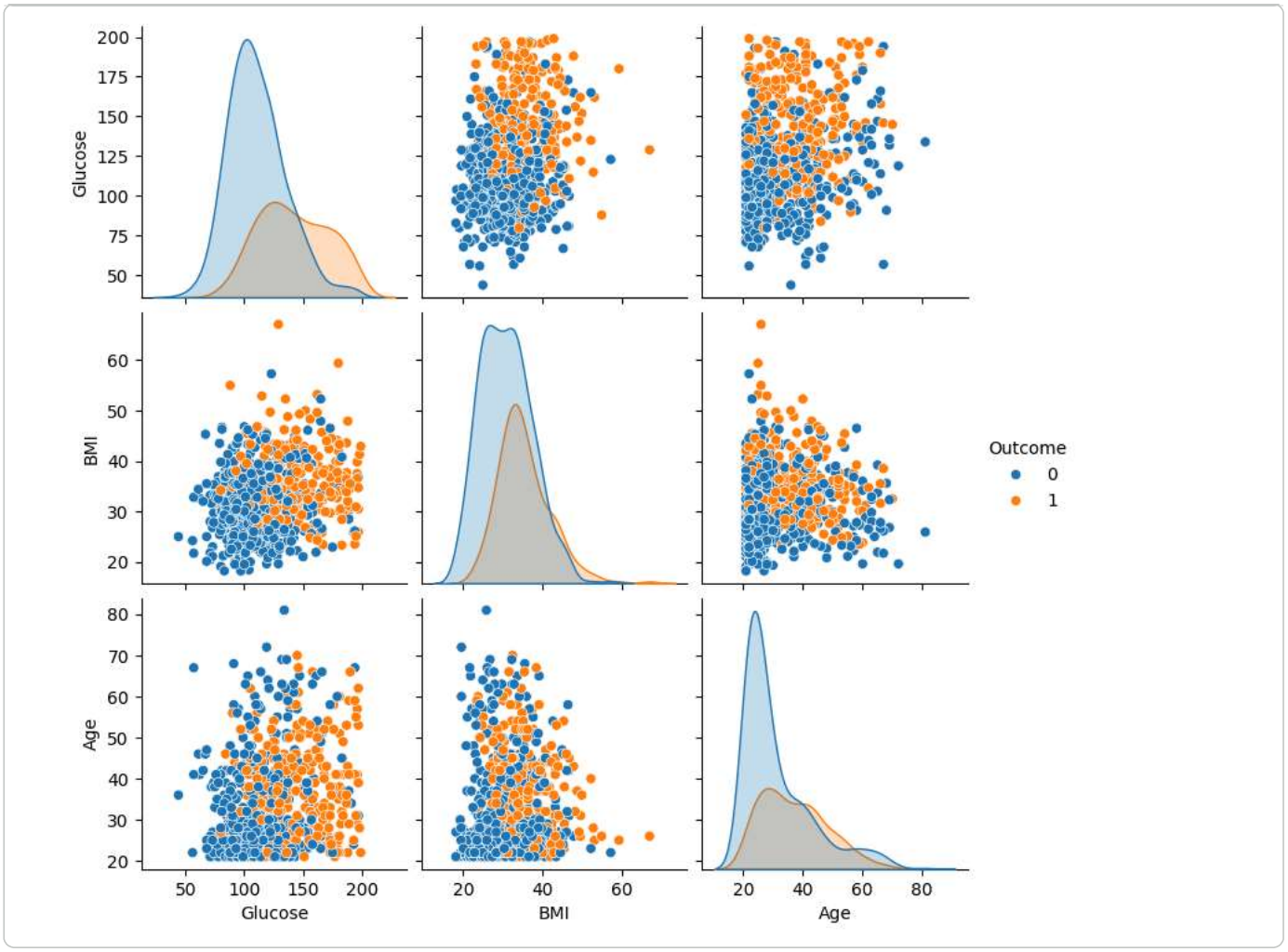
Correlation of the all columns using Heatmap plot

```
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



Pairplot

```
# Pairplot (subset)
sns.pairplot(df[['Glucose', 'BMI', 'Age', 'Outcome']], hue='Outcome')
plt.show()
```



Split Data for Training and testing

```
# separating the data and labels
X = df.drop(columns = 'Outcome', axis=1)
y = df['Outcome']
```

```
X = X.astype(float)
```

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
```

Train_Test Data

```
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, train_size=0.8, random_state=42, stratify=y_resam
```

Normalization(Scalling)

```
Start coding or generate with AI.
```

scalar = StandardScaler() X_train = scalar.fit_transform(X_train) X_test = scalar.fit_transform(X_test)

print(X_train) print('----------') print(X_test)

Bulid Multiple Models

```
# Models

models = {
    "Random Forest": RandomForestClassifier(random_state= 42),
    "SVM": SVC(random_state=42),
    "xgboost": XGBClassifier(random_state =42)
}

accuracy_scores = {}
```

```
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    accuracy_scores[name] = accuracy_score(y_test, pred)

accuracy_scores
```

```
{'Random Forest': 0.815, 'SVM': 0.74, 'xgboost': 0.8}
```

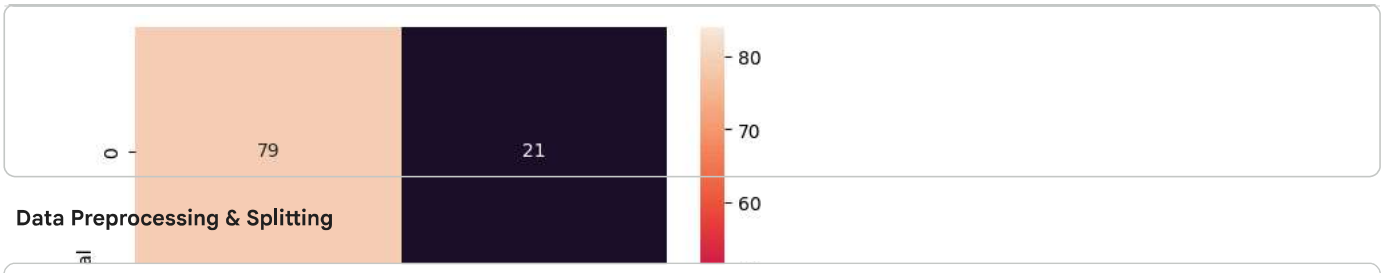
```
best_model = max(accuracy_scores, key=accuracy_scores.get)
best_accuracy = accuracy_scores[best_model]

print("Best Model:", best_model)
print("Accuracy:", best_accuracy)
import pickle
pickle.dump(best_model, open('diabetes.pkl', 'wb'))
```

```
Best Model: Random Forest
Accuracy: 0.815
```

```
final_model = models[best_model]
y_pred = final_model.predict(X_test)

cm = confusion_matrix(y_true=y_test, y_pred=y_pred)
cr = classification_report(y_true=y_test, y_pred=y_pred)
print(cm)
print(cr)
```

```
[[79 21]
 [16 84]]
              precision    recall  f1-score   support

           0       0.83      0.79      0.81       100
           1       0.80      0.84      0.82       100

    accuracy                           0.81       200
   macro avg       0.82      0.81      0.81       200
weighted avg       0.82      0.81      0.81       200
```

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



Data Preprocessing & Splitting

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop('Outcome', axis=1)
y = df['Outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, accuracy_score

# Models ki list
models = {
    "SVM": SVC(kernel='linear'),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='logloss')
}

# Loop chala kar results nikalna
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print(f"\n--- {name} Performance ---")
    print(f"Accuracy: {accuracy_score(y_test, predictions):.2%}")
    print(classification_report(y_test, predictions))
```

```
--- SVM Performance ---
Accuracy: 76.62%
              precision    recall  f1-score   support

           0       0.79      0.86      0.83        99
           1       0.70      0.60      0.65        55
```